

WEEK 2 — SECURITY IMPLEMENTATION REPORT

Securing the NodeGoat Application

Student Name	Fatima Haq
Course	Cybersecurity
Project Title	Securing the NodeGoat Application
Date	June 26, 2026
Security Controls Implemented	4 (Validation, Hashing, JWT, Helmet)

CONFIDENTIAL – For Educational / Internship Use Only

1. Objective

The objective of this task was to improve the security of the vulnerable NodeGoat application by implementing security mechanisms such as input validation, password hashing, JWT authentication, and secure HTTP headers. Each control directly addresses a vulnerability identified during the Week 1 security assessment.

2. Summary of Implemented Controls

Security Control	Library Used	Status	OWASP Category
Input Validation & Sanitization	validator	Implemented	A03:2021 – Injection
Password Hashing	bcryptjs / bcrypt	Implemented	A02:2021 – Cryptographic Failures
JWT Authentication	jsonwebtoken	Implemented (generation)	A07:2021 – Auth Failures
Secure HTTP Headers	helmet	Implemented	A05:2021 – Security Misconfiguration

3. Before vs. After Comparison

Area	Before	After
Email input	Any string accepted, no validation	Rejected unless RFC-valid format (validator.isEmail)
Password storage	Stored in plain text in MongoDB	Stored as bcrypt hash, e.g. \$2a\$10\$RR.ed9DJ...
Session/auth	No token issued after login	Signed JWT issued, expires in 1 hour
HTTP headers	No security headers; X-Powered-By exposed Express	Helmet headers applied; X-Powered-By removed

4. Implementation Details

Task 1 — Input Validation and Sanitization

Objective

To validate user inputs and prevent invalid or malicious data from being submitted to the application.

Implementation

The validator library was installed:

```
npm install validator
```

The library was imported into session.js:

```
const validator = require("validator");
```

Server-side email validation was implemented:

```
if (email !== "") {
  if (!validator.isEmail(email)) {
    errors.emailError = "Invalid email address";
    return false;
  }
}
```

Note: Validation here runs only on the server. Client-side validation (e.g. HTML5 input type="email" or a JS check before submit) should also be added for better user experience — but server-side validation is the one that actually matters for security, since client-side checks can be bypassed.

Testing

- Invalid email addresses such as abc@gmail were entered.

The application displayed the message:

```
Invalid email address
```

- Valid email addresses were accepted successfully.

Result

The application now validates email addresses on the server side and rejects invalid input.

Task 2 — Password Hashing

Objective

To securely store user passwords in the database using hashing.

Implementation

A bcrypt-compatible hashing package was installed:

```
npm install bcrypt
```

Note: Your install command and code both use the bcrypt / bcryptjs API (hashSync, genSaltSync, compareSync). The earlier mention of "bcrypt-nodejs" refers to a different package with a different API and has been corrected here to bcrypt for accuracy — make sure your package.json reflects whichever one you actually installed.

Passwords are hashed before being stored in MongoDB. Implementation in user-dao.js:

```
password: bcrypt.hashSync(password, bcrypt.genSaltSync())
```

During login, passwords are verified using:

```
bcrypt.compareSync(plainPassword, hashedPassword)
```

Note: genSaltSync() with no argument defaults to 10 salt rounds. It is good practice to set this explicitly, e.g. genSaltSync(12), so the cost factor is documented and consistent across environments.

Testing

- New users were registered.

Passwords stored in MongoDB appeared in hashed format, for example:

```
$2a$10$RR.ed9DJqyQVTE1cAgSYauKXdScLnY5ckbPOH8aBvaMiBm/oTaE6.
```

- Login functionality was tested successfully with the hashed credentials.

Result

Passwords are no longer stored in plain text. Sensitive user credentials are protected using secure hashing.

Task 3 — JWT Authentication

Objective

To implement token-based authentication.

Implementation

The jsonwebtoken package was installed:

```
| npm install jsonwebtoken
```

The package was imported:

```
| const jwt = require("jsonwebtoken");
```

A JWT token is generated after successful login:

```
| const token = jwt.sign(  
|   { id: user._id },  
|   "your-secret-key",  
|   { expiresIn: "1h" }  
| );
```

Note: The secret key is hardcoded in this example. In a real or production-grade implementation, the secret must be moved to an environment variable (e.g. process.env.JWT_SECRET) and never committed to source control — a hardcoded secret defeats the purpose of signing, since anyone with the code can forge valid tokens.

Note: This implementation covers token generation only. To make the authentication functional end-to-end, a verification middleware (using jwt.verify()) should also be added to protected routes, so the server actually checks the token on each request rather than just issuing it at login. This is recommended as a next step.

Testing

After successful login, the terminal generated a JWT token:

```
| JWT Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Result

The application now generates authentication tokens after successful login.

Task 4 — Securing HTTP Headers using Helmet.js

Objective

To improve the security of HTTP responses by adding secure headers.

Implementation

Helmet was installed:

```
| npm install helmet
```

Helmet middleware was enabled in server.js:

```
| const helmet = require("helmet");  
  
| app.use(helmet());  
| app.disable("x-powered-by");
```

Testing

The application was executed successfully after enabling Helmet middleware. Security headers were automatically added to HTTP responses, and the X-Powered-By header (which previously revealed the Express framework) was removed.

Result

The application now uses secure HTTP headers, reducing risks such as clickjacking and information disclosure.

5. Limitations & Recommended Next Steps

The following items are recommended as follow-up work beyond the scope of this task:

- Move the JWT secret key to an environment variable instead of hardcoding it in source code.
- Add JWT verification middleware to protect authenticated routes (e.g. /profile, /memos).
- Set an explicit bcrypt salt round count (e.g. 12) rather than relying on the default.
- Extend input validation to other fields (username, memo content) in addition to email.
- Re-run the OWASP ZAP scan from Week 1 to confirm the previously identified findings are resolved.

6. Conclusion

In this task, several important security mechanisms were implemented in the NodeGoat application. Input validation, password hashing, JWT authentication, and secure HTTP headers significantly improved the overall security posture of the application. These security enhancements help protect the application against common web vulnerabilities and strengthen authentication and data protection mechanisms.

Combined with the findings from the Week 1 assessment, this work directly addresses three of the highest-priority risks identified earlier: plain text password storage, missing security headers, and lack of input validation. JWT-based authentication lays the foundation for stronger session management, pending the remaining verification work noted above.

This report was produced as part of a cybersecurity internship training exercise on a deliberately vulnerable application in a local environment.